ReconX v3 — Active Recon & Web Mapping Implementation Plan

Builds on: the OSINT provider-expansion work already shipped (provider registry, circuit breaker, config schema v2, concurrency ceiling). **Source:** Web Recon Automation Tool Blueprint (19 core modules). **Stack:** Python 3.12+, Typer, httpx + asyncio, Playwright, BeautifulSoup4, dnspython, Jinja2 + WeasyPrint, SQLite, Rich + Loguru.

Ground Rule Change vs Plan 1

Plan 1 (OSINT expansion) was 100% passive — public API reads only. This plan introduces **active recon against the target itself**: direct HTTP requests, DNS lookups, TLS handshakes, and a headless-browser crawl of the target's own pages. This is still safe for an authorized target (you're just visiting the site like a browser would), but it is a different trust boundary than querying third-party OSINT APIs. Keep the two paths architecturally separate:

- `internal/providers/` (or `reconx/providers/`) → third-party OSINT lookups (Plan 1)
- `reconx/modules/` → direct-to-target modules (this plan)

Both should register into the **same provider/module registry** (`ProviderMeta`-style) so `reconx providers list` / `reconx modules list` show one unified picture, and both share the circuit breaker + concurrency ceiling you already built.

Phase 1 — Core Recon (foundation, do first)

These modules replace what you'd otherwise be tempted to hardcode ad hoc, and everything later (crawler, tech detection, reporting) depends on their output schema.

Module	What	Key output fields	Notes
Target Processing	Normalize input URL → scheme, host, port, resolved IP(s)	`scheme, host, port, ips[]`	Handle bare domain, IP, and full URL input uniformly. Reject/flag private-IP targets unless `--allow-internal` is passed (safety net).
DNS	A, AAAA, MX, TXT, SPF, DKIM (common selectors), DMARC, NS, SOA, CNAME	one dict per record type	Use `dnspython`, async resolver. DKIM needs a selector guess list (`default`, `google`, `selector1/2`, `k1`) — no universal way to enumerate it.
SSL/TLS	Cert chain, issuer, SANs, validity window, cipher suite, TLS version, HSTS header	`cert{}, ciphers[], hsts_max_age`	Use `ssl` + `cryptography` for parsing; don't reinvent ASN.1 parsing. Flag expired/self-signed/weak-cipher as findings, not just data.
WHOIS/ASN	Registrar, creation/expiry date, ASN, org, prefix	`whois{}, asn{}`	Reuse BGPView/Team Cymru from Plan 1 Batch 3 instead of a separate WHOIS library where possible — avoids duplicate rate-limit surfaces.
HTTP headers/cookies/redirects	Full response headers, `Set-Cookie` list, redirect chain with status codes	`headers{}, cookies[], redirect_chain[]`	Follow redirects manually (don't let httpx auto-follow silently) so you can log the chain and detect redirect loops/open redirects later.

Why this order: DNS resolution feeds target processing, TLS needs the resolved host, and headers/redirects need a live connection — this is a strict dependency chain, not

parallelizable on first run.

Hands-on task: Run core recon against a domain with DNSSEC, a redirect chain of 3+ hops, and an HSTS header. Confirm all five modules populate correctly and the SQLite row has no null fields where data should exist.

Expected result: One consolidated JSON blob per target with all five modules' output, timestamped, before any web-mapping module runs.

Common mistake: Doing TLS analysis and HTTP header fetch as two separate connections. Reuse the same `httpx.AsyncClient` session/connection where you can — testers auditing your tool will notice double connection overhead in a packet capture.

Phase 2 — Web Mapping (the active crawl)

This is the highest-effort phase. Everything here depends on Playwright.

2.1 Website Crawler

- **What:** BFS crawl from the root URL, same-origin by default, configurable depth (`--max-depth`) and page cap (`--max-pages`).
- **Why:** Feeds every downstream module (forms, JS, endpoints, auth) — build this before them, not alongside.
- **How:** Playwright for JS-rendered pages (don't rely on BeautifulSoup alone — modern sites won't expose links in raw HTML). Dedup by normalized URL (strip tracking params, fragment).
- **Common mistake:** No same-origin check → crawler wanders off-target into third-party CDNs/social widgets. Always scope to the target's registered domain (and explicitly opt-in for subdomains).

2.2 robots.txt / sitemap.xml / security.txt

- **What:** Fetch and parse all three from well-known paths.
- **Why:** robots.txt often reveals disallowed (interesting) paths; sitemap.xml gives you a crawl seed list for free; security.txt gives contact/disclosure info for the report.
- **How:** Simple GET + parse (`robotparser` stdlib for robots.txt). Feed sitemap URLs into the crawler's seed queue.

2.3 Form Detection

- **What:** Extract every `<form>` from crawled pages: action URL, method, input fields, field types, hidden fields, CSRF token presence.
- **Why:** Precondition for both authentication detection and later (manual) testing.
- **Output:** one row per form: `page_url, action, method, fields[], has_csrf_token`.

2.4 Authentication Detection

- **What:** Heuristic classification of login/register/password-reset forms (field name patterns: `password`, `email`+`password` pair, `2fa`/`otp` fields) plus detection of auth-related response headers (`WWW-Authenticate`, session cookie flags: `HttpOnly`, `Secure`, `SameSite`).
- **Why:** Flags attack surface without doing anything active (no login attempts) — stays passive-to-the-target's-auth-logic.
- **Common mistake:** Don't auto-submit forms during detection. Detection = static/DOM analysis only; leave actual auth testing to the manual VAPT phase.

2.5 API Discovery

- **What:** Two sources: (a) JS bundle scraping for endpoint-shaped strings (`/api/`, `/v1/`, `/graphql`), (b) Playwright network-request capture during the crawl (record every XHR/fetch call the page makes).
- **Why:** (b) is far higher signal than regex-on-JS — it catches what the page *actually calls*, not just string literals.
- **Output:** deduped endpoint list with method, observed status code, content-type.

2.6 File Upload/Download Discovery

- **What:** Flag forms with `enctype="multipart/form-data"` or file inputs (upload); flag links to common download extensions (`.pdf, .zip, .exe, .xlsx`, etc.) or `Content-Disposition: attachment` responses (download).
- **Why:** Both are common finding categories in a VAPT report — worth a category of their own rather than burying them in "forms" and "links."

2.7 JavaScript Analysis

- **What:** Pull all `<script>` src files (first-party and third-party, tagged separately), scan for hardcoded secrets/API keys (basic regex set: AWS keys, generic `api_key=`, JWT-looking strings), and source-map leak detection (`.js.map` reachable).
- **Why:** Cheap, high-value finding category; source maps left in prod are a very common real-world miss.
- **Common mistake:** Don't try to fully deobfuscate/minify-reverse JS here — that's a rabbit hole. Regex-based secret scanning + source-map presence check is enough for this tool's scope; flag for manual review, don't claim to be a full static analyzer.

Hands-on task for Phase 2: Crawl a test app with a login form, a file upload page, and at least one XHR-driven API call. Confirm the tool captures the API call via network interception (not just JS regex) and correctly classifies the login form.

Expected result: A `webmap.json` with crawled pages, forms, detected APIs, JS findings, and upload/download points — all cross-referenced by page URL.

Phase 3 — Security Configuration Analysis

Runs against data already collected in Phases 1–2; mostly a rules-engine layer, not new network calls.

Module	Checks
Security headers	Presence/correctness of `CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy`, HSTS with `preload`/`includeSubDomains`
Cookie analysis	`Secure`, `HttpOnly`, `SameSite` flags on every cookie seen during the crawl; flag session cookies missing any of the three
CDN/WAF detection	Header fingerprints (`cf-ray`→Cloudflare, `x-akamai-*`, `server: cloudflare`, etc.) + response-timing/behavior heuristics as fallback

Why grouped here: This is your **findings layer** — it should output a severity-tagged list (`info/low/medium/high`), same shape as your Plan 1 category rollup, so Phase 5 reporting can render it consistently.

Common mistake: Hardcoding a small header-presence check without a real ruleset. Base the CSP check on actual weak-policy patterns (`unsafe-inline`, `*` wildcard sources), not just "header exists."

Phase 4 — AI Business Analyzer

- **What:** Feed the consolidated recon data (tech stack, page titles/content, detected APIs) to an LLM to produce a plain-English business-context summary: likely industry, apparent purpose of the site, notable third-party integrations, and a prioritized narrative of the top findings.
- **Why:** This is the one module that turns a data dump into something a client/report reader actually wants to read first.
- **How:** One structured-JSON-output prompt (system prompt forces JSON-only response) using your existing recon JSON as input. Keep this as a clearly-labeled "AI-generated summary" section in the report — don't let it silently blend with your factual findings.
- **Common mistake:** Sending the full raw JS/HTML dump to the LLM. Send the structured findings only (tech stack list, header findings, endpoint list) — cheaper, more reliable, and avoids prompt-injection risk from content scraped off the target site itself. Treat scraped page text as untrusted input to the LLM call.

Phase 5 — Reporting

- **What:** HTML (Jinja2 templates) → PDF (WeasyPrint) → JSON (raw export) from one shared data model.
- **Structure:** Executive summary (AI analyzer output) → Infrastructure (DNS/WHOIS/ASN/TLS) → Web Mapping (crawl stats, forms, APIs) → Security Findings (severity-grouped, same category-rollup pattern as Plan 1) → Screenshots → Appendix (raw JSON).
- **Screenshot capture:** Playwright full-page screenshot per crawled page (or top N by depth) — store as files, reference by path in the report, don't inline as base64 in HTML (bloats file size fast).
- **Common mistake:** Building three separate report generators (HTML/PDF/JSON) with duplicated logic. Build one data model → Jinja2 template → render to HTML → WeasyPrint HTML-to-PDF → and a `.json()` dump of the same model. One source of truth, three renders.

Phase 6 — Plugin Architecture (ties Plan 1 + Plan 2 together)

- **What:** Formalize `reconx/modules/` (active, this plan) and `reconx/providers/` (passive OSINT, Plan 1) under one plugin interface: each registers `name, category, requires_target_contact (bool), run(target) -> Finding[]`.
- **Why:** `requires_target_contact` is the key flag that separates "safe against any target" (OSINT) from "touches the target directly" (this plan) — surface this distinction in `reconx modules list` so a user auditing scope can see exactly what will and won't hit the target.
- **Integration point:** Both plugin types feed the same `Finding` schema → same severity/category rollup → same report generator from Phase 5.

Suggested Timeline

Week	Work
1	Phase 1 (Core Recon) — DNS, TLS, WHOIS/ASN, headers
2	Phase 2.1–2.2 (crawler + robots/sitemap/security.txt)
3	Phase 2.3–2.6 (forms, auth detection, API discovery, upload/download)
4	Phase 2.7 (JS analysis) + Phase 3 (security header/cookie/CDN rules engine)
5	Phase 5 (reporting: HTML/PDF/JSON, screenshots)
6	Phase 4 (AI business analyzer) — do this after reporting exists, so you have a report to slot it into
7	Phase 6 (unify plugin interface with Plan 1 providers) + README pass

Cross-Cutting Common Mistakes

1. Building the crawler and every downstream module in parallel — the crawler is a hard dependency; build and verify it alone first.
2. Letting the AI analyzer see raw scraped page content instead of structured findings (prompt-injection risk from the target's own pages).
3. Treating active modules and OSINT modules as the same trust tier — always keep `requires_target_contact` visible so scope discussions with a client are unambiguous.
4. Base64-inlining screenshots into HTML reports — kills load time and file size at scale.
5. Skipping the shared `Finding` schema and writing bespoke output shapes per module — you'll redo Phase 5 twice if you do.

Next Step

Start with Phase 1 (Core Recon) since every later module consumes its output. Want the Python module skeleton (dataclasses for `Finding`, `ProviderMeta`-equivalent `ModuleMeta`, and the async runner) sized to slot into your existing `reconx/` package structure?